Práctica 3: Java RMI

Desarrollo de Sistemas Distribuidos. Grupo DSD-3.

Grado en Ingeniería Informática

Curso 2025-2026.

ETSIIT. Universidad de Granada.





UNIVERSIDAD
DE GRANADA

Autor:
Ángel Rodríguez Faya

Profesor:
José Ángel Segura Muros

Fecha de entrega:
24 de abril de 2026

Índice

1. Introducción **2**

2. Parte 1. Implementación de Ejemplos. **2**

2.1. Ejemplo 1. Cliente-Servidor básico. 2

2.1.1. `Ejemplo_I.java` (La Interfaz Remota) 3

2.1.2. `Ejemplo.java` (El Servidor) 3

2.1.3. `Cliente_Ejemplo.java` (El Cliente) 5

2.1.4. `launch.sh` (Script de Ejecución) 5

2.1.5. Diagrama de clases del ejemplo 1. 8

2.1.6. Ejemplo de compilación y ejecución del ejemplo 1. . . . 8

2.2. Ejemplo 2. Multihebra. 10

2.2.1. `Ejemplo_I.java` (La Interfaz Remota) 10

2.2.2. `Ejemplo.java` (El Servidor) 10

2.2.3. `Cliente_Ejemplo_Multi_Threaded.java` (El Cliente) . 12

2.2.4. `launch.sh` (Script de Ejecución) 13

2.2.5. Diagrama de clases del ejemplo 2. 14

2.2.6. Ejemplo de compilación y ejecución del ejemplo 2. Respondiendo a las preguntas 15

2.3. Ejemplo 3. Contador. 18

2.3.1. `icontador.java` (La Interfaz Remota) 18

2.3.2. `contador.java` (El Objeto Remoto) 19

2.3.3. `servidor.java` (El Servidor) 19

2.3.4. `cliente.java` (El Cliente) 20

2.3.5. `launch.sh` (Script para Ejecución) 21

2.3.6. Diagramas en UML del ejemplo 3. 22

2.3.7. Ejemplo de compilación y ejecución del ejemplo 3. . . . 24

3. Parte 2. Servidor Replicado. **25**

3.1. Ejemplo 1. Cliente-Servidor básico. 25

1. Introducción

En esta tercera práctica vamos a trabajar y aprender el manejo de los mecanismos para implementar programas distribuidos en Java, haciendo uso de su API **RMI** (***R**emote **M**ethod **I**nvocation*), que nos permite construir objetos distribuidos. Como entorno de funcionamiento se utilizará la máquina virtual de java (*JVM*, Java Virtual Machine) que sigue un enfoque *"Write Once, Run Anywhere"*, que su traducción viene a ser: "se codifica una vez y se ejecuta en cualquier sitio"), usando este enfoque para que se ejecute en ó por todas partes (*"run everywhere"*).

Los sistemas RMI se caracterizan porque existe un objeto delegado (que llamaremos *stub)*) que es el encargado de gestionar las invocaciones al objeto remoto, gestionando la serialización y deserialización de los parámetros y valores de retorno.

Las aplicaciones en RMI (denominadas **aplicaciones de objetos distribuidos**)están formadas por dos aplicaciones: el cliente y el servidor. El servidor es el encargado de crear los objetos remotos, hacer accesibles las referencias a dichos objetos remotos y esperar la invocación de métodos sobre dichos objetos remotos por parte de los clientes. Estos, los clientes, obtienen una referencia remota a uno o varios objetos remotos en el servidor e invocan métodos sobre los objetos remotos. RMI proporciona los mecanismos que permiten la comunicació y el intercambio de información entre el cliente y el servidor.

A continuación, veremos los tres ejemplos que nos dan en el guion de este práctica, los ejecutaremos y analizaremos que está sucediendo, así como mostrar sus archivos, pues considero que es más cómodo a la hora de corregir, tenerlo todo en un documento (hablo de tener el código fuente de los ejercicios más ejemplos de ejecución), que tener la necesidad de abrir el documento y el editor de código a la vez.

2. Parte 1. Implementación de Ejemplos.

2.1. Ejemplo 1. Cliente-Servidor básico.

Este ejemplo es una muestra de cómo se realiza una implementación básica de una aplicación Cliente-Servidor en Java RMI. Tenemos un servidor llamado `Ejemplo` que implementa un método `escribir_mensaje(int idProceso)`, que recibe un entero que representa el identificador del proceso (Cliente) que invoca a este método del servidor, y muestra por pantalla dicho identificador. Si se trata del proceso/hebra 0, entonces la duerme 5 segundos

y después continua como el resto de procesos.

Veamos cada uno de los archivos que forman este ejemplo. Es importante señalar que **este código ha sido modificado durante las sesiones de prácticas**, pues se trata de una práctica antigua y hay paquetes/componentes que actualmente están en estado obsoletos.

2.1.1. `Ejemplo_I.java` (La Interfaz Remota)

Es la interfaz que define qué métodos del servidor pueden ser llamados por el cliente a través de la red.

Su código es:

```
1  // Ejemplo_I.java
2  // Define la Interfaz remota
3
4  import java.rmi.Remote;
5  import java.rmi.RemoteException;
6
7  public interface Ejemplo_I extends Remote {
8      public void escribir_mensaje (int id_proceso) throws RemoteException;
9  }
```

2.1.2. `Ejemplo.java` (El Servidor)

Implementa la interfaz anteriormente definida, es decir, el método `escribir_mensaje`. Esta aplicación está a la espera de recibir peticiones. Aquí se han **realizado modificaciones**, ya que el `SecurityManager` del ejemplo está obsoleto (*deprecated*) a partir de Java 17, por lo que se han añadido las clases `SslRMIClientSocketFactory` y `SslRMIServerSocketFactory` para que la comunicación vaya cifrada a través de **SSL/TSL**.

Su código es:

```
1  // Ejemplo.java
2  // Implementa la Interfaz remota
3
4  import java.rmi.registry.LocateRegistry;
5  import java.rmi.registry.Registry;
6  import java.rmi.server.UnicastRemoteObject;
7  import javax.rmi.ssl.SslRMIClientSocketFactory;
8  import javax.rmi.ssl.SslRMIServerSocketFactory;
9
10 public class Ejemplo implements Ejemplo_I {
11     private static final int RMI_PORT = 2099;
12
```

```
13      public Ejemplo() {
14          super();
15      }
16
17      public void escribir_mensaje (int id_proceso) {
18          System.out.println("Recibida peticion de proceso: "+id_proceso);
19          if (id_proceso == 0) {
20              try{
21                  System.out.println("Empezamos a dormir");
22                  Thread.sleep(5000);
23                  System.out.println("Terminamos de dormir");
24              }
25              catch (Exception e) {
26                  System.err.println("Ejemplo exception:");
27                  e.printStackTrace();
28              }
29          }
30          System.out.println("\nHebra "+id_proceso);
31      }
32
33      public static void main(String[] args) {
34          try {
35              // Definimos el nombre del objeto remoto que se registrará en el ligador
36              String nombre_objeto_remoto = "Ejemplo_I";
37
38              // Creamos una instancia del objeto remoto
39              Ejemplo_I prueba = new Ejemplo();
40
41              // Creamos los socket factories para SSL
42              SslRMIClientSocketFactory csf = new SslRMIClientSocketFactory();
43              SslRMIServerSocketFactory ssf = new SslRMIServerSocketFactory();
44
45              // Creamos el stub del objeto remoto para el cliente, exportandolo al
46              // puerto 0 (puerto anonimo) y usando los socket factories creados.
47              Ejemplo_I stub =
48                  (Ejemplo_I) UnicastRemoteObject.exportObject(prueba, 0, csf, ssf);
49
50              // El Registry vive como proceso externo (rmiregistry).
51              Registry registry = LocateRegistry.createRegistry(RMI_PORT);
52
53              // Registramos el objeto remoto en el ligador
54              registry.rebind(nombre_objeto_remoto, stub);
55
56              // A partir de aquí, el objeto remoto ya puede ser invocado por los clientes.
57
58              System.out.println("Ejemplo bound");
59          }
60          catch (Exception e) {
61              System.err.println("Ejemplo exception:");
```

```
            e.printStackTrace();
        }
    }
}
```

2.1.3. `Cliente_Ejemplo.java` (El Cliente)

Es el encargado de conectar con el servidor para realizar peticiones. Para ello, se conecta al registro (el `rmiregistry`) que hace de guía para poder encontrar nuestros objetos, y busca el servidor por su nombre, que sería ".Ejemplo_I". Después se llama al método para escribir el mensaje, pasándole su identificador de proceso como parámetro.

Su código sería el siguiente:

```
// Cliente_Ejemplo.java
// Código del cliente que invoca al objeto remoto implementado en Ejemplo.java

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Cliente_Ejemplo {
    private static final int RMI_PORT = 2099;

    public static void main(String args[]) {
        try {
            String nombre_objeto_remoto = "Ejemplo_I";
            System.out.println("Buscando el objeto remoto");
            Registry registry = LocateRegistry.getRegistry(args[0], RMI_PORT);
            // Invocamos el objeto remoto obtenido del ligador
            Ejemplo_I instancia_local = (Ejemplo_I) registry.lookup(nombre_objeto_remoto);
            System.out.println("Invocando el objeto remoto");
            // Llamamos al método remoto, pasándole
            // el id del proceso (recibido como argumento)
            instancia_local.escribir_mensaje(Integer.parseInt(args[1]));
        } catch (Exception e) {
            System.err.println("Ejemplo_I exception:");
            e.printStackTrace();
        }
    }
}
```

2.1.4. `launch.sh` (Script de Ejecución)

Nos han proporcionado este script para poder realizar la compilación/ejecución con tan sólo ejecutarlo. Básicamente, lo que hace es crear

unos “certificados digitales” necesarios para que el SSL funcione en neustra máquina, compilar todos los .java de golpe, lanzar el servidor en segundo plano y por último lanzar dos clientes para que se conecten al servidor, estos clientes tienen como identificadores 0 y 3, respectivamente.

```
#!/bin/sh -e
# ejecutar = Macro para compilación y ejecución del programa ejemplo
# en una sola maquina Unix de nombre localhost.

KEYSTORE=server-keystore.p12
TRUSTSTORE=client-truststore.p12
STOREPASS=changeit
RMI_PORT=2099

if [ ! -f "$KEYSTORE" ]; then
        echo
        echo "Generando keystore TLS de ejemplo ..."
        keytool -genkeypair \
                -alias rmi-server \
                -keyalg RSA \
                -keysize 2048 \
                -validity 3650 \
                -dname "CN=localhost, OU=RMI, O=DSD, L=Local, ST=Local, C=ES" \
                -storetype PKCS12 \
                -keystore "$KEYSTORE" \
                -storepass "$STOREPASS" \
                -keypass "$STOREPASS" \
                -noprompt
fi

if [ ! -f "$TRUSTSTORE" ]; then
        echo
        echo "Generando truststore TLS de ejemplo ..."
        keytool -exportcert \
                -alias rmi-server \
                -keystore "$KEYSTORE" \
                -storetype PKCS12 \
                -storepass "$STOREPASS" \
                -rfc \
                -file rmi-server.crt

        keytool -importcert \
                -alias rmi-server \
                -file rmi-server.crt \
                -storetype PKCS12 \
                -keystore "$TRUSTSTORE" \
                -storepass "$STOREPASS" \
                -noprompt
```

```
44
45          rm -f rmi-server.crt
46  fi
47
48  # Si quedo un proceso previo en el puerto, lo cerramos para evitar conflictos.
49  if command -v fuser >/dev/null 2>&1; then
50          fuser -k "${RMI_PORT}/tcp" >/dev/null 2>&1 || true
51  fi
52
53  echo
54  echo "Compilando con javac ..."
55  javac *.java
56
57  sleep 2
58
59  echo
60  echo "Lanzando el servidor"
61  java -cp . \
62          -Djava.rmi.server.hostname=localhost \
63          -Djavax.net.ssl.keyStore="$KEYSTORE" \
64          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
65          -Djavax.net.ssl.keyStoreType=PKCS12 \
66          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
67          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
68          -Djavax.net.ssl.trustStoreType=PKCS12 \
69          Ejemplo &
70
71  sleep 2
72
73  echo
74  echo "Lanzando el primer cliente"
75  echo
76  java -cp . \
77          -Djavax.net.ssl.keyStore="$KEYSTORE" \
78          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
79          -Djavax.net.ssl.keyStoreType=PKCS12 \
80          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
81          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
82          -Djavax.net.ssl.trustStoreType=PKCS12 \
83          Cliente_Ejemplo localhost 0
84
85  sleep 2
86
87  echo
88  echo "Lanzando el segundo cliente"
89  echo
90  java -cp . \
91          -Djavax.net.ssl.keyStore="$KEYSTORE" \
92          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
```

```
93          -Djavax.net.ssl.keyStoreType=PKCS12 \
94          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
95          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
96          -Djavax.net.ssl.trustStoreType=PKCS12 \
97          Cliente_Ejemplo localhost 3
```

2.1.5. Diagrama de clases del ejemplo 1.

El diagrama de clases en UML del ejemplo 1 es:

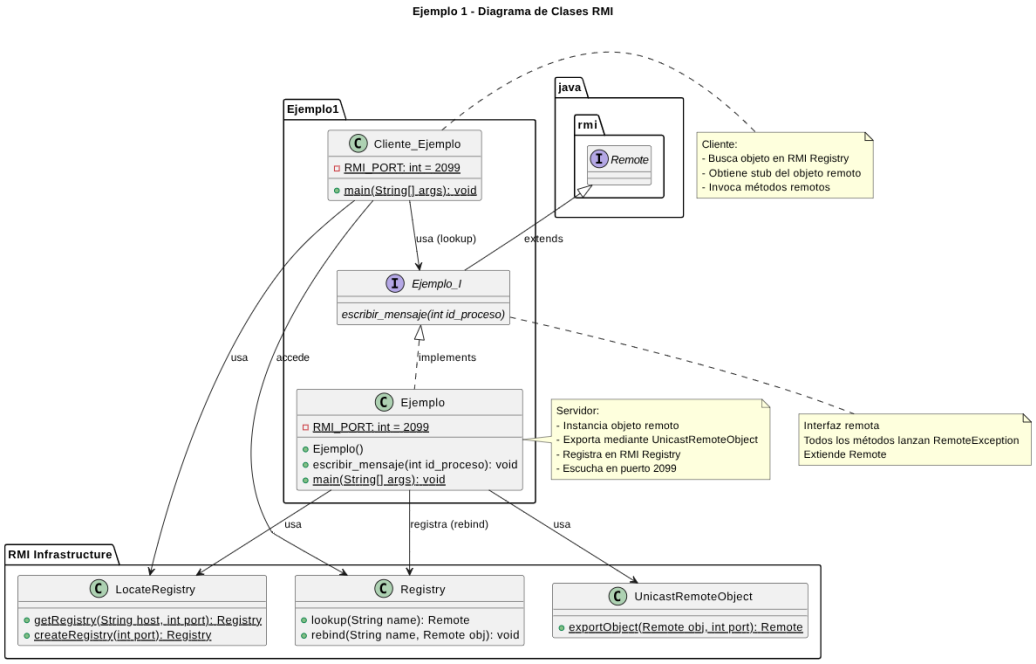


Figura 1: Diagrama de clases del ejemplo 1

2.1.6. Ejemplo de compilación y ejecución del ejemplo 1.

En primer lugar, debemos dar permiso de ejecución al script. Para ello abrimos una terminal y ejecutamos el siguiente comando:

```
$ chmod u+x launch.sh
```

Después, ejecutamos el script con el siguiente comando:

```
$ ./launch.sh
```

A continuación, muestro una imagen con la salida de la ejecución:

```
angelrf@angel-ubuntu:~/Practica_3_RMI/codigo/Parte_1/Ejemplo_1$ chmod u+x launch.sh
angelrf@angel-ubuntu:~/Practica_3_RMI/codigo/Parte_1/Ejemplo_1$ ./launch.sh

Generando truststore TLS de ejemplo ...
Certificado almacenado en el archivo <rmi-server.crt>
Se ha agregado el certificado al almacén de claves

Compilando con javac ...

Lanzando el servidor
Ejemplo bound

Lanzando el primer cliente

Buscando el objeto remoto
Invocando el objeto remoto
Recibida peticion de proceso: 0
Empezamos a dormir
Terminamos de dormir

Hebra 0

Lanzando el segundo cliente

Buscando el objeto remoto
Invocando el objeto remoto
Recibida peticion de proceso: 3

Hebra 3
```

Figura 2: Ejecución del ejemplo 1

Como podemos observar en la salida, se compilan todos los archivos, se lanza el servidor y se muestra el mensaje `.Ejemplo bound"`, que nos indica que el objeto remoto ya puede ser invocado por los clientes. Después se lanza el cliente 0. Este busca el objeto remoto y lo invoca. El método de escribir mensaje en el servidor recibe la petición mostrando ese mensaje con el id del proceso cliente. Como se trata del proceso 0, lo pone a dormir 5 segundos. Pasado ese tiempo, muestra el mensaje de que ha terminado de dormir e imprime rl mensaje de hebra con el id de nuevo, y ya termina. Finalmente se lanza el cliente con id 3 (aunque no se lanza después de que termine la hebra 0, si no que **se lanza la hebra 3 mientras la hebra 0 duerme**).De nuevo, este cliente busca el objeto remoto y lo invoca. Llama al método para escribir mensaje, este muestra que le ha llegado la petición mostrando su id.

2.2. Ejemplo 2. Multihebra.

Partiendo del ejemplo anterior, vamos a realizar una modificación y es que no vamos a lanzar varios clientes como habíamos hecho en el ejemplo anterior, si no que vamos a crear múltiples hebras dentro de un mismo cliente. Estas hebras van a realizar la misma tarea, que será imprimir un mensaje remoto accediendo al *stub* de un objeto remoto. El objetivo de este ejemplo es dejar constancia de la gestión de concurrencia de RMI.

De nuevo, vamos a ver los archivos y comentar los cambios que se han realizado con respecto a la versión del ejemplo 1.

2.2.1. `Ejemplo_I.java` (La Interfaz Remota)

Se ha modificado el parámetro que recibe el método imprimir mensaje, que ahora es un `String` en lugar de un entero.

Su código es:

```
1  // Ejemplo_I.java
2  // Define la Interfaz remota
3
4  import java.rmi.Remote;
5  import java.rmi.RemoteException;
6
7  public interface Ejemplo_I extends Remote {
8      public void escribir_mensaje (String mensaje) throws RemoteException;
9  }
```

2.2.2. `Ejemplo.java` (El Servidor)

Se cambia la implementación del método `escribir_mensaje`. En este caso, se informa de cuando entra y cuando sale la hebra, cuyo nombre es el mensaje pasado como parámetro al método. Además, **las hebras que sean múltiplo de 10 (su nombre termina en 0)**, se dormirán durante 5 segundos

Su código es:

```
1  // Ejemplo.java
2  // Implementa la Interfaz remota
3
4  import java.rmi.registry.LocateRegistry;
5  import java.rmi.registry.Registry;
6  import java.rmi.server.UnicastRemoteObject;
7  import javax.rmi.ssl.SslRMIClientSocketFactory;
8  import javax.rmi.ssl.SslRMIServerSocketFactory;
```

```
9
10 public class Ejemplo implements Ejemplo_I {
11     private static final int RMI_PORT = 2099;
12
13     public Ejemplo() {
14         super();
15     }
16
17     public synchronized void escribir_mensaje(String mensaje) {
18         System.out.println("\nEntra Hebra " + mensaje);
19         //Buscamos los procesos 0, 10, 20,...
20         if (mensaje.endsWith("0")) {
21             try {
22                 System.out.println("Empezamos a dormir");
23                 Thread.sleep(5000);
24                 System.out.println("Terminamos de dormir");
25             } catch (Exception e) {
26                 System.err.println("Ejemplo exception:");
27                 e.printStackTrace();
28             }
29         }
30         System.out.println("Sale Hebra " + mensaje);
31     }
32
33     public static void main(String[] args) {
34         try {
35             // Definimos el nombre del objeto remoto que se registrará en el ligador
36             String nombre_objeto_remoto = "Ejemplo_I";
37
38             // Creamos una instancia del objeto remoto
39             Ejemplo_I prueba = new Ejemplo();
40
41             // Creamos los socket factories para SSL
42             SslRMIClientSocketFactory csf = new SslRMIClientSocketFactory();
43             SslRMIServerSocketFactory ssf = new SslRMIServerSocketFactory();
44
45             // Creamos el stub del objeto remoto para el cliente, exportandolo al
46             // puerto 0 (puerto anonimo) y usando los socket factories creados.
47             Ejemplo_I stub =
48                 (Ejemplo_I) UnicastRemoteObject.exportObject(prueba, 0, csf, ssf);
49
50             // El Registry vive como proceso externo (rmiregistry).
51             Registry registry = LocateRegistry.createRegistry(RMI_PORT);
52
53             // Registramos el objeto remoto en el ligador
54             registry.rebind(nombre_objeto_remoto, stub);
55
56             // A partir de aquí, el objeto remoto ya puede ser invocado por los clientes.
57
```

```
58              System.out.println("Ejemplo bound");
59          }
60          catch (Exception e) {
61              System.err.println("Ejemplo exception:");
62              e.printStackTrace();
63          }
64      }
65  }
```

2.2.3. `Cliente_Ejemplo_Multi_Threaded.java` (El Cliente)

Esta clase representa el cliente. Es distinta con respecto a la del ejercicio 1. Esta clase `Cliente_Ejemplo_Multi_Threaded` implementa la interfaz `Runnable` que nos permite crear *threads* o hebras en Java. Uno de los cambios es que ahora se le pasa el host del servidor como parámetro en el constructor de la clase, almacenándolo en un atributo. Como implementa esta interfaz la interfaz `Runnable`, tiene que darle una implementación al método `run()`, que será ejecutado por cada una de las hebras. En este método, cada hebra busca su propio stub y llama al servidor (al objeto remoto) de forma concurrente con el resto de hebras creadas.

En el `main`, se extrae el número de hebras, que se pasa como argumento al programa. Se crean tantas hebras como se hayan indicado, pasándoles el nombre del servidor al crearlas e indicándole su nombre, que será `Cliente + i`, donde i será un entero entre 0 y el número de hebras - 1. Las hebras se guardarán en un array.

Su código es:

```
1   // Cliente_Ejemplo_Multi_Threaded.java
2   // Código del cliente que invoca al objeto remoto implementado en Ejemplo.java
3
4   import java.rmi.registry.LocateRegistry;
5   import java.rmi.registry.Registry;
6
7   public class Cliente_Ejemplo_Multi_Threaded implements Runnable {
8       private static final int RMI_PORT = 2099;
9       public String nombre_objeto_remoto = "Ejemplo_I";
10      public String server;
11
12      public Cliente_Ejemplo_Multi_Threaded(String server) {
13          // Almacenamos el nombre del host del servidor
14          this.server = server;
15      }
16
17      public void run() {
18          System.out.println("Buscando el objeto remoto");
```

```
19          try {
20              Registry registry = LocateRegistry.getRegistry(server, RMI_PORT);
21              // Invocamos el objeto remoto obtenido del ligador
22              Ejemplo_I instancia_local = (Ejemplo_I) registry.lookup(nombre_objeto_remoto);
23              System.out.println("Invocando el objeto remoto");
24              // Llamamos al método remoto, pasándole
25              // el id del proceso (recibido como argumento)
26              instancia_local.escribir_mensaje(Thread.currentThread().getName());
27          } catch (Exception e) {
28              System.err.println("Ejemplo_I exception:");
29              e.printStackTrace();
30          }
31      }
32
33      public static void main(String args[]) {
34          int n_hebras = Integer.parseInt(args[1]);
35          Cliente_Ejemplo_Multi_Threaded[] v_clientes = new Cliente_Ejemplo_Multi_Threaded[n_hebi
36          Thread[] v_hebras = new Thread[n_hebras];
37          for (int i = 0; i < n_hebras; i++) {
38              //A cada hebra le pasamos el nombre el servidor
39              v_clientes[i] = new Cliente_Ejemplo_Multi_Threaded(args[0]);
40              v_hebras[i] = new Thread(v_clientes[i], "Cliente " + i);
41              v_hebras[i].start();
42          }
43      }
44  }
```

2.2.4. `launch.sh` (Script de Ejecución)

Simplemente se ha modificado para lanzar el servidor y el cliente correctamente. Al cliente se le indica que cree 12 hebras.

Las únicas líneas que han cambiado con respecto al ejemplo 1 son:

```
58
59  echo
60  echo "Lanzando el servidor Ejemplo 2"
61  java -cp . \
62          -Djava.rmi.server.hostname=localhost \
63          -Djavax.net.ssl.keyStore="$KEYSTORE" \
64          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
65          -Djavax.net.ssl.keyStoreType=PKCS12 \
66          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
67          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
68          -Djavax.net.ssl.trustStoreType=PKCS12 \
69          Ejemplo &
70
71  sleep 2
```

```
72
73  echo
74  echo "Lanzando el cliente multi-hebra (12 hebras)"
75  java -cp . \
76          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
77          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
78          -Djavax.net.ssl.trustStoreType=PKCS12 \
79          Cliente_Ejemplo_Multi_Threaded localhost 12
```

2.2.5. Diagrama de clases del ejemplo 2.

El diagrama de clases en UML del ejemplo 2 es:

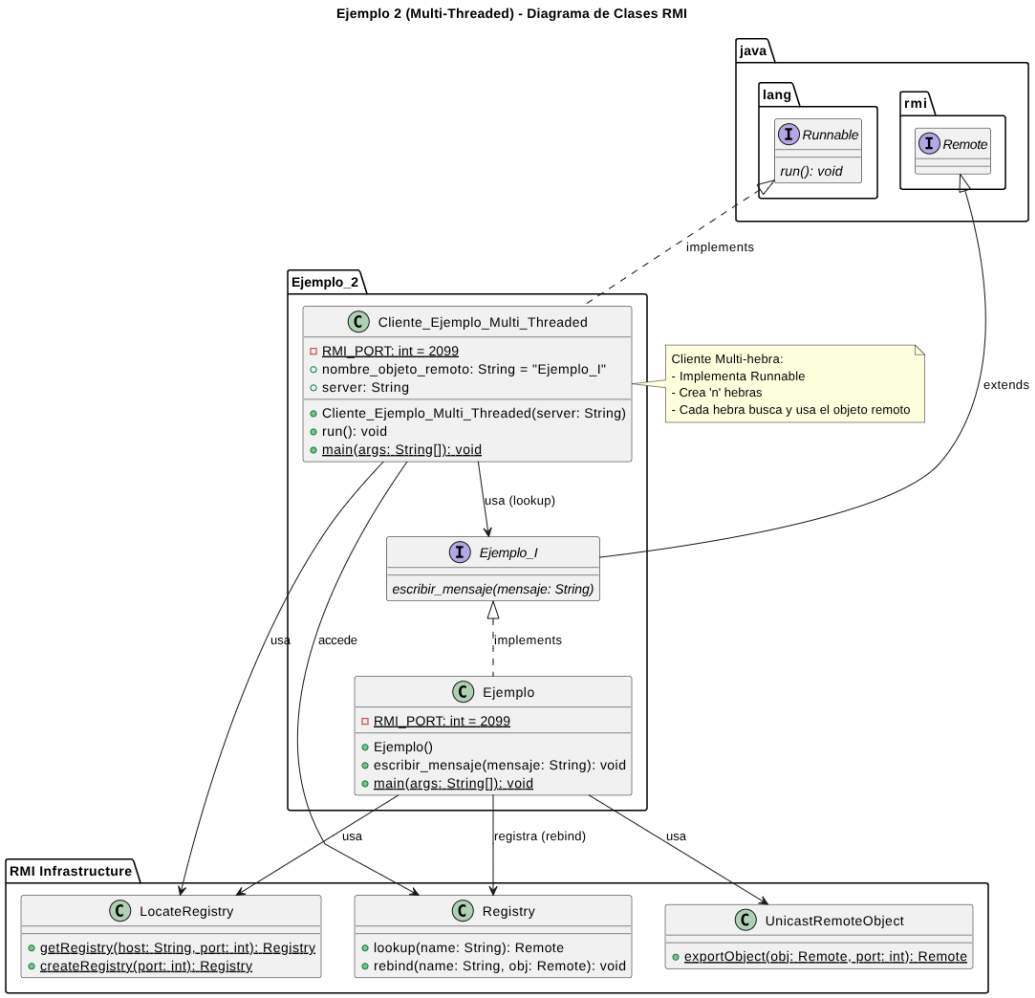


Figura 3: Diagrama de clases del ejemplo 2

2.2.6. Ejemplo de compilación y ejecución del ejemplo 2. Respondiendo a las preguntas

De nuevo, damos permiso de ejecución al script y lo ejecutamos. La salida sería:

```
angelrf@angel-ubuntu:~/Practica_3_RMI/codigo/Parte_1/Ejemplo_2$ chmod u+x launch.sh

angelrf@angel-ubuntu:~/Practica_3_RMI/codigo/Parte_1/Ejemplo_2$ ./launch.sh


Compilando con javac ...
Lanzando el servidor Ejemplo 2
Ejemplo bound
Lanzando el cliente multi-hebra (22 hebras)
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto

Entra Hebra Cliente 2
Sale Hebra Cliente 2

Entra Hebra Cliente 8
Sale Hebra Cliente 8

Entra Hebra Cliente 9
Sale Hebra Cliente 9

Entra Hebra Cliente 7
Sale Hebra Cliente 7

Entra Hebra Cliente 5
Sale Hebra Cliente 5

Entra Hebra Cliente 4
Sale Hebra Cliente 4

Entra Hebra Cliente 10
Empezamos a dormir

Entra Hebra Cliente 0
Empezamos a dormir

Entra Hebra Cliente 11
Sale Hebra Cliente 11

Entra Hebra Cliente 6
Sale Hebra Cliente 6

Entra Hebra Cliente 3
Sale Hebra Cliente 3
Invocando el objeto remoto

Entra Hebra Cliente 1
Sale Hebra Cliente 1
Terminamos de dormir
Sale Hebra Cliente 0
Terminamos de dormir
Sale Hebra Cliente 10
```

Figura 4: 1ª ejecución del ejemplo 2

Nos piden que comprobemos que ocurre con las hebras cuyo nombre acaba en 0 y en que se diferencian del resto. Pues, como ya hemos comentado anteriormente, las hebras cuyo nombre es múltiplo de 10, el método remoto `escribir_mensaje` las duerme durante 5 segundos.

Luego nos indican que prestemos atención a la impresión de mensajes y comprobemos si estos mensajes se entrelazan. La respuesta es SÍ. Si nos fijamos en el ejemplo de ejecución que muestro arriba, podemos observar al final del todo que, las hebras 0 y 10 (que son las que tienen que dormir puesto que son las únicas que terminan en 0 de las 12 hebras que hay en total), entran al método pero se quedan dormidas, y el resto de hebras conforme entran al método, salen de forma instantánea. Mientras que, estas dos hebras, es al final, pasados esos 5 segundos, cuando despiertan y ya terminan. Con esto podemos ver como las hebras se ejecutan y llaman al método remoto (realizan peticiones/llamadas al servidor) de forma concurrente.

Vamos a hacer la **2ª modificación que nos pide el guion.** Voy a introducir el modificador `synchronized` en el método de la implementación remota y vamos a ver un ejemplo de ejecución y vamos a analizar las diferencias con respecto a la ejecución anterior.

¡Ojo! Esta modificación se realiza en la clase donde se implementa el método, NO en la interfaz donde se define. En nuestro caso, sería en `Ejemplo.java`. El método quedaría como:

```
public synchronized void escribir_mensaje (String mensaje)
throws RemoteException;
```

Volvemos a ejecutar el script con: `$ ./launch.sh` y vemos que su salida es:



Figura 5: 2ª ejecución del ejemplo 2

Vamos a explicar que esta sucediendo. Al añadir la palabra `synchronized` **estamos convertimos todo el método en una sección crítica. Java asocia a cada objeto un monitor o cerrojo (*lock*)** como los que estudiamos en la asignatura de *Sistemas Distribuidos y Concurrentes*. Entonces si hay varias clases que quieren acceder a una clase o método, sólo puede acceder una a la vez y modificarla, y así no pueden modificarlas todas de golpe. Es por eso que, cuando una hebra entra en el método remoto `escribir_mensaje` adquiere automáticamente el cerrojo de la instancia del objeto `Ejemplo`. Hasta que la hebra no abandona el método, no se libera el cerrojo. Es por eso que, cuando entra la hebra 0 o 10, y se ponen a dormir esos 5 segundos, las hebras que vienen detrás tendrán que esperar también esos 5 segundos (entrando en un estado bloqueo), hasta que la hebra múltiplo de 10 se despierte y abandone el método, liberando así el cerrojo.

2.3. Ejemplo 3. Contador.

En este ejemplo sobre un contador vamos a crear **por una parte el objeto remoto**, que está formado por varias funciones accesibles de forma remota; **y por otra parte el servidor**, implementado en el archivo `servidor.java`, que exporta los métodos contenidos en la interfaz `icontador.java` del objeto remoto instanciado como `micontador` de la clase definida en `contador.java`.

Por otro lado encontramos el **programa cliente**, que es una aplicación en Java y realiza las siguientes acciones: inicializar el contador del servidor, invocar el método `incrementar` del contador unas 1.000 veces, e imprimir el valor final del contador junto con el tiempo de respuesta medio que se ha calculado a partir de las invocaciones remotas del método incrementar.

De nuevo, comentar que los archivos han sido levemente modificados para evitar los problemas de componentes obsoletos. Vamos a verlos:

2.3.1. `icontador.java` (La Interfaz Remota)

Declara los métodos que tendrá el objeto remoto que la implemente.

```
1  // Fichero: icontador.java
2  // Define la Interfaz remota
3
4  import java.rmi.Remote;
5  import java.rmi.RemoteException;
6
```

```
7  public interface icontador extends Remote {
8
9      int sumar() throws RemoteException;
10
11     void sumar(int valor) throws RemoteException;
12
13     public int incrementar() throws RemoteException;
14 }
```

2.3.2. `contador.java` (El Objeto Remoto)

Es el objeto remoto que implementa la interfaz remota, y por lo tanto, le da una implementación a los métodos declarados en dicha interfaz. Con respecto al ejemplo que nos proporciona el guion, se ha quitado el `extends UnicastRemoteObject` para exportarlo manualmente en el servidor con SSL.

```
1  // Fichero: contador.java
2  // Es el objeto remoto que implementa la interfaz remota
3
4  import java.rmi.RemoteException;
5
6  public class contador implements icontador {
7      private int suma;
8
9      public contador() {
10         suma = 0;
11     }
12
13     public int sumar() throws RemoteException {
14         return suma;
15     }
16
17     public void sumar(int valor) throws RemoteException {
18         suma = valor;
19     }
20
21     public int incrementar() throws RemoteException {
22         suma++;
23         return suma;
24     }
25 }
```

2.3.3. `servidor.java` (El Servidor)

Es el servidor encargado de publicar el objeto remoto anterior.

```java
// Fichero: servidor.java
// Es el servidor que publica el objeto remoto contador

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.server.UnicastRemoteObject;
import javax.rmi.ssl.SslRMIClientSocketFactory;
import javax.rmi.ssl.SslRMIServerSocketFactory;

public class servidor {
    public static void main(String[] args) {
        try {
            int puerto = 1099;
            contador micontador = new contador();

            // Configuramos las factorías SSL
            SslRMIClientSocketFactory csf = new SslRMIClientSocketFactory();
            SslRMIServerSocketFactory ssf = new SslRMIServerSocketFactory();

            // Creamos el stub del objeto remoto para el cliente, exportándolo
            // al puerto 0 (puerto anónimo) y usando las factorías SSL
            icontador stub = (icontador) UnicastRemoteObject.exportObject(micontador, 0, csf,

            // Creamos el registro en el puerto 1099
            Registry registry = LocateRegistry.createRegistry(puerto);
            registry.rebind("mmicontador", stub);

            System.out.println("Servidor contador preparado con SSL en puerto " + puerto);
        } catch (Exception e) {
            System.err.println("Excepción en el servidor: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

2.3.4. `cliente.java` (El Cliente)

Implementa el cliente. Sus funciones ya se comentaron anteriormente.

```java
// Fichero: cliente.java
// Es el cliente que invoca al objeto remoto contador

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class cliente {
    public static void main(String[] args) {
```

```
9           if (args.length < 1) {
10              System.out.println("Uso: java cliente <host>");
11              System.exit(1);
12          }
13
14          try {
15              // Crea el stub para el cliente especificando el nombre del servidor
16              // y el puerto del registro RMI
17              Registry mireg = LocateRegistry.getRegistry(args[0], 1099);
18
19              icontador micontador = (icontador) mireg.lookup("mmicontador");
20
21              // Pone el contador al valor inicial 0
22              System.out.println("Poniendo contador a 0...");
23              micontador.sumar(0);
24
25              // Obtiene hora de comienzo
26              long horacomienzo = System.currentTimeMillis();
27
28              // Incrementa 1000 veces
29              System.out.println("Incrementando 1000 veces...");
30              for (int i = 0; i < 1000; i++) {
31                  micontador.incrementar();
32              }
33              // Obtiene hora final, realiza e imprime calculos
34              long horafin = System.currentTimeMillis();
35              System.out.println("Media de las RMI realizadas = "
36                              + ((horafin - horacomienzo) / 1000f)
37                              + " msegs");
38              System.out.println("Valor final del contador = " + micontador.sumar());
39
40          } catch (Exception e) {
41              System.err.println("Excepción del sistema: " + e);
42          }
43          System.exit(0);
44      }
45  }
```

2.3.5. `launch.sh` (Script para Ejecución)

Se ha modificado la forma en la que se lanza el cliente y el servidor. Las únicas líneas que han cambiado con respecto a los ejemplos anteriores son:

```
58
59  echo "Lanzando Servidor Contador (Ejemplo 3)"
60  java -cp . \
```

```
61      -Djava.rmi.server.hostname=localhost \
62      -Djavax.net.ssl.keyStore="$KEYSTORE" \
63      -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
64      -Djavax.net.ssl.keyStoreType=PKCS12 \
65      -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
66      -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
67      -Djavax.net.ssl.trustStoreType=PKCS12 \
68      servidor &
69
70  sleep 2
71
72  echo "Lanzando Cliente Contador"
73  java -cp . \
74      -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
75      -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
76      -Djavax.net.ssl.trustStoreType=PKCS12 \
77      cliente localhost
```

2.3.6. Diagramas en UML del ejemplo 3.

El **diagrama de clases** en UML de este ejemplo es:

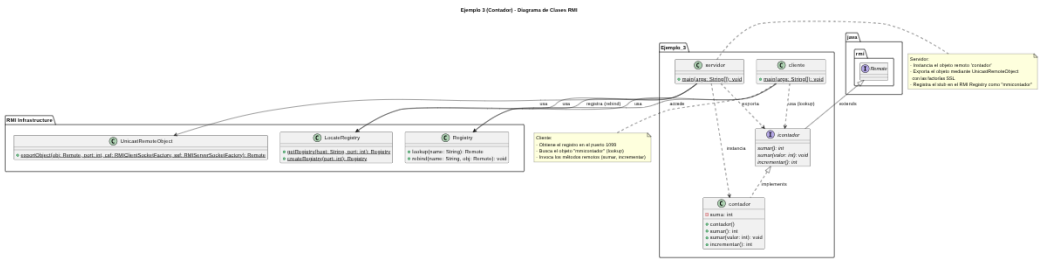


Figura 6: Diagrama de clases en UML del ejemplo 3

Y también muestro a continuación el **diagrama de secuencia** en UML de este ejemplo, ya que considero qué es relevante ya que hemos separado el objeto remoto del servidor.

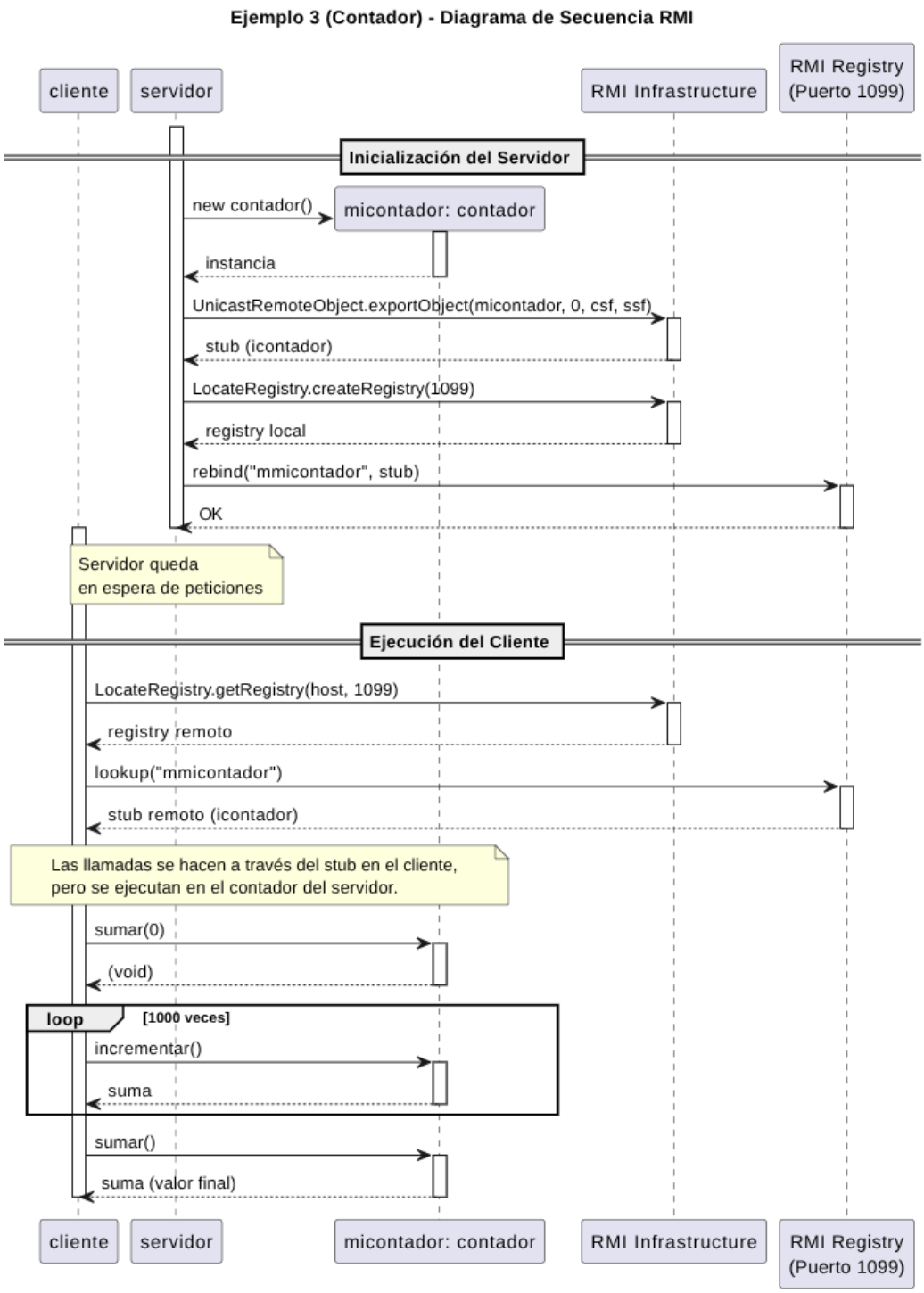


Figura 7: Diagrama de secuencia en UML del ejemplo 3

2.3.7. Ejemplo de compilación y ejecución del ejemplo 3.

Una vez más, al igual que hicimos en los ejemplos anteriores, damos permiso de ejecución al script y lo ejecutamos.

La salida sería:

```
angelrf@angel-ubuntu:~/Practica_3_RMI/codigo/Parte_1/Ejemplo_3$ chmod u+x launch.sh
angelrf@angel-ubuntu:~/Practica_3_RMI/codigo/Parte_1/Ejemplo_3$ ./launch.sh

Generando keystore TLS de ejemplo ...
Generando par de claves RSA de 2.048 bits para certificado autofirmado (SHA384withRSA) con una
 validez de 3.650 días
        para: CN=localhost, OU=RMI, O=DSD, L=Local, ST=Local, C=ES

Generando truststore TLS de ejemplo ...
Certificado almacenado en el archivo <rmi-server.crt>
Se ha agregado el certificado al almacén de claves

Compilando con javac ...
Lanzando Servidor Contador (Ejemplo 3)
Servidor contador preparado con SSL en puerto 1099
Lanzando Cliente Contador
Poniendo contador a 0...
Incrementando 1000 veces...
Media de las RMI realizadas = 0.233 msegs
Valor final del contador = 1000
```

Figura 8: Ejecución del ejemplo 3

Vamos a comentar varias cosas que podemos deducir tras ver el código del ejemplo y la imagen de la salida (Figura 8):

- **El servidor recuerda el valor de suma.** Si lanzamos el cliente varias veces sin reiniciar el servidor (y quitamos la línea de `micontador.sumar(0)`), podremos ver que el contador sigue subiendo. El objeto remoto vive en el servidor mientras este esté en ejecución.
- Al realizar 1000 llamadas seguidas, **el cliente calcula el tiempo medio de una invocación RMI**, que lo podemos ver en la figura 8. **Llamar al método remoto es mucho más lento que una llamada a un método local**, porque cada incremento implica:
  - Serializar el nombre del método.
  - Enviar el paquete por la red (TCP/SSL).
  - Procesarlo en el servidor.
  - Recibir la confirmación o respuesta.
- Vemos que hay una **gran transparencia**, ya que el cliente usa el método `incrementar()` como si fuera un objeto local, pero el cálculo real ocurre en la JVM del servidor.

3. Parte 2. Servidor Replicado.

3.1. Ejemplo 1. Cliente-Servidor básico.